

Reviewer Pack — p=3 Path-Family Modulus Lower-Bounds Tseitin Tree-Resolution

Entry 67ed17c63684 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 10:26:58 UTC

p=3 Path-Family Modulus Lower-Bounds Tseitin Tree-Resolution

Entry ID: 67ed17c63684 Verdict: INCONCLUSIVE Recorded: 2026-05-18 10:26:58 UTC

1. Conjecture

Field A (mathematical branch): Discrete quasiconformal analysis — p-modulus of path families on weighted graphs for $p \neq 2$ (Beurling 1946 modulus tradition; Heinonen-Koskela 1998 ‘Quasiconformal maps in metric spaces of controlled geometry’; Albin-Brunner-Perez-Pisaroni-Poggi-Corradini 2015 ‘Modulus of families of walks on graphs’; Albin-Poggi-Corradini’s discrete-modulus school). The $p=3$ functional $\sum_e \rho(e)^3$ is convex but NOT a quadratic form, so it lies strictly between min-cut ($p=1$) and effective resistance ($p=2$), with a dual that is a Hölder-3 flow-cost program rather than a Sherman-Morrison pseudoinverse identity. Distinct from blacklisted effective-resistance / signed-Laplacian / Szegedy-walk / Ihara-zeta attempts: none of those is a $p \neq 2$ modulus. Uses sup/abs/cube-of-sums (no polynomial F_q extension, Aaronson-Wigderson-safe). An arXiv search ‘p-modulus’ AND (‘Tseitin’ OR ‘resolution refutation’ OR ‘DPLL’ OR ‘proof complexity’) returns 0 direct hits and <5 adjacent papers (all on Albin-Poggi-Corradini network modulus or QC analysis on fractals, never on CNF refutation). **Field B** (complexity object): Tree-like Resolution / DPLL refutation tree size $t(T(G, \omega))$ for Tseitin XOR formulas $T(G, \omega)$ on connected 3-regular graphs G with odd charge ω (Urquhart 1987 / Ben-Sasson-Wigderson 2001 / Alekhnovich-Razborov 2001 expansion regime), in the Cook-Reckhow-Krajíček bounded-arithmetic framework where $\neg T(G, \omega)$ is Σ^b_0 and $\log_2 t$ lower-bounds V^0 -witnessing complexity. The invariant is STRUCTURAL on (G, ω) : two charged graphs whose Tseitin formulas are both $\equiv \text{FALSE}$ can have very different M_3 , shielding from Razborov-Rudich natural proofs.

Statement:

Let G be a connected 3-regular graph on n vertices, $m=3n/2$ edges, and let $\omega: V(G) \rightarrow \mathbb{F}_2$ satisfy $\sum_v \omega(v)=1$ (odd-charge, so $T(G, \omega)$ is unsatisfiable). Let Γ_ω be the family of simple G -paths whose endpoints $u \neq v$ both have $\omega(u)=\omega(v)=1$. Define the discrete $p=3$ modulus $M_3(G, \omega) := \inf \{ \sum_e \rho(e)^3 : \rho: E(G) \rightarrow \mathbb{R}_{\geq 0} \text{ and } \sum_{e \in \gamma} \rho(e) \geq 1 \text{ for all } \gamma \in \Gamma_\omega \}$, computed in polynomial time by convex programming. Conjecture: there exists an absolute constant $c>0$ such that for every such (G, ω) , $\log_2 t^*(T(G, \omega)) \geq c \cdot n \cdot M_3(G, \omega)$; furthermore for every (G, ω) with G a 3-regular expander (girth ≥ 4 , $\lambda_2(A_G) \geq 0.4$) and ω odd, $M_3(G, \omega) \geq 1/(c \cdot n^{\{2/3\}})$. A single (G, ω) violating either inequality with the calibrated $c=0.05$ refutes the conjecture.

Rationale (proposer’s reasoning):

Tseitin hardness on expanders is governed by edge-disjoint flow-thickness between charged vertices, which the $p=2$ (effective resistance) modulus already captures only up to spectral information. The $p=3$ functional penalises concentrated densities CUBICALLY, so on expanders the optimal ρ is spread nearly uniformly, giving $M_3 \approx m \cdot m^{-\{3\}}$

$\approx \Theta(n^{\{-2/3\}})$; on bottlenecked instances M_3 collapses toward a single-edge cost ≈ 1 . This non-quadratic response is invisible to all spectral / pseudoinverse / non-backtracking invariants already exhausted on Tseitin, and the cube-of-sums dual is precisely an $L^{\{3/2\}}$ -flow program — a regime never touched by the Ben-Sasson-Wigderson width / expansion machinery.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 04e2619bfa74e099

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 10, 12, 16\}$ with 30 accepted seeds each (120 instances), conjecture is SUPPORTED iff min over all instances of $\log_2(t^*)/(n \cdot M_3) \geq 0.05$ AND min of $n^{\{2/3\}} \cdot M_3 \geq 0.05$, with mean ratios ≥ 0.10 . Any single instance below 0.05 on either metric FALSIFIES.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-modulus path family graph Tseitin resolution lower bound - discrete p-modulus proof complexity DPLL refutation expander - Albin Poggi-Corradini modulus walks Tseitin CNF tree resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import time
from collections import deque, defaultdict
from fractions import Fraction

def matrix_mult(A, B):
    n = len(A)
```

```

m = len(B[0])
p = len(B)
result = [[0 for _ in range(m)] for _ in range(n)]
for i in range(n):
    for j in range(m):
        for k in range(p):
            result[i][j] += A[i][k] * B[k][j]
return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_add(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_sub(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_scalar_mult(A, scalar):
    return [[A[i][j] * scalar for j in range(len(A[0]))] for i in range(len(A))]

def matrix_trace(A):
    return sum(A[i][i] for i in range(min(len(A), len(A[0]))))

def matrix_norm(A):
    return math.sqrt(sum(sum(a**2 for a in row) for row in A))

def matrix_inverse(A):
    n = len(A)
    I = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
    for col in range(n):
        diag_row = max(range(col, n), key=lambda r: abs(A[r][col]))
        A[col], A[diag_row] = A[diag_row], A[col]
        I[col], I[diag_row] = I[diag_row], I[col]
        if A[col][col] == 0:
            raise ValueError("Matrix is not invertible")
        for i in range(col + 1, n):
            factor = Fraction(A[i][col], A[col][col])
            for j in range(col, n):
                A[i][j] -= factor * A[col][j]
            for j in range(n):
                I[i][j] -= factor * I[col][j]
    for col in reversed(range(n)):
        for i in reversed(range(col)):
            factor = Fraction(A[i][col], A[col][col])
            for j in range(n):
                A[i][j] -= factor * A[col][j]
                I[i][j] -= factor * I[col][j]
    for i in range(n):
        factor = Fraction(1, A[i][i])
        for j in range(n):
            I[i][j] *= factor
    return I

def is_connected(adj):
    n = len(adj)

```

```

visited = [False] * n
stack = [0]
visited[0] = True
count = 1
while stack:
    u = stack.pop()
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            stack.append(v)
            count += 1
return count == n

def girth(adj):
    n = len(adj)
    min_girth = float('inf')
    for u in range(n):
        visited = [-1] * n
        queue = deque([(u, 0)])
        visited[u] = 0
        while queue:
            v, dist = queue.popleft()
            for w in adj[v]:
                if visited[w] == -1:
                    visited[w] = dist + 1
                    queue.append((w, dist + 1))
                elif w != u and visited[w] >= dist:
                    cycle_length = visited[w] + dist + 1
                    if cycle_length < min_girth:
                        min_girth = cycle_length
    return min_girth if min_girth != float('inf') else 0

def spectral_gap(adj):
    n = len(adj)
    D = [[0]*n for _ in range(n)]
    for i in range(n):
        D[i][i] = len(adj[i])
    L = matrix_sub(D, adj)
    try:
        L_inv = matrix_inverse(L)
        eigs = [matrix_trace(matrix_mult(L, L_inv)) / n]
        return 1 - eigs[0]
    except:
        return 0

def sample_3_regular_graph(n, seed):
    random.seed(seed)
    while True:
        edges = []
        stubs = list(range(n)) * 3
        random.shuffle(stubs)
        for i in range(0, len(stubs), 2):
            u, v = stubs[i], stubs[i+1]
            if u != v and (u, v) not in edges and (v, u) not in edges:
                edges.append((u, v))
        adj = [[] for _ in range(n)]

```

```

    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    if is_connected(adj) and girth(adj) >= 4 and spectral_gap(adj) >= 0.4:
        return adj

def sample_odd_charge(n, seed):
    random.seed(seed)
    omega = [random.randint(0, 1) for _ in range(n)]
    if sum(omega) % 2 == 0:
        omega[random.randint(0, n-1)] ^= 1
    return omega

def find_simple_paths(adj, omega, K=80):
    n = len(adj)
    paths = []
    charged_vertices = [v for v in range(n) if omega[v] == 1]
    for u in charged_vertices:
        for v in charged_vertices:
            if u != v:
                visited = [False] * n
                queue = deque([(u, [u])])
                visited[u] = True
                count = 0
                while queue and count < K:
                    current, path = queue.popleft()
                    for neighbor in adj[current]:
                        if neighbor == v:
                            paths.append(path + [neighbor])
                            count += 1
                        elif not visited[neighbor]:
                            visited[neighbor] = True
                            queue.append((neighbor, path + [neighbor]))

    return paths

def compute_M3(adj, omega, paths, max_iter=1500):
    m = len(adj) * 3 // 2
    rho = [Fraction(1, m) for _ in range(m)]
    lambda_val = 1
    for _ in range(max_iter):
        grad = [3 * rho[i]**2 for i in range(m)]
        for path in paths:
            path_edges = set()
            for i in range(len(path)-1):
                u, v = path[i], path[i+1]
                if u > v:
                    u, v = v, u
                edge_idx = (u * (2*m - u - 1) // 2 + v - u - 1) // 2
                path_edges.add(edge_idx)
            sum_rho = sum(rho[i] for i in path_edges)
            if sum_rho < 1:
                for i in path_edges:
                    grad[i] += 2 * lambda_val * (1 - sum_rho)
        step_size = 0.01
        new_rho = [max(0, rho[i] - step_size * grad[i]) for i in range(m)]
        if sum((new_rho[i] - rho[i])**2 for i in range(m)) < 1e-6:

```

```

        break
    rho = new_rho
    lambda_val *= 1.1
    return sum(rho[i]**3 for i in range(m))

def tseitin_tree_resolution(adj, omega, max_nodes=500000):
    n = len(adj)
    clauses = []
    for u in range(n):
        neighbors = adj[u]
        for i in range(len(neighbors)):
            for j in range(i+1, len(neighbors)):
                v, w = neighbors[i], neighbors[j]
                clauses.append((u, v, w))
    variables = [f'x_{u}_{v}' for u in range(n) for v in adj[u]]
    assignments = {}
    nodes_explored = 0

    def backtrack():
        nonlocal nodes_explored
        if nodes_explored >= max_nodes:
            return float('inf')
        nodes_explored += 1
        if all(omega[u] == sum(1 for v in adj[u] if assignments.get(f'x_{u}_{v}', 0)) % 2 for u in range(n)):
            return 0
        for clause in clauses:
            u, v, w = clause
            x_uv = assignments.get(f'x_{u}_{v}', None)
            x_uw = assignments.get(f'x_{u}_{w}', None)
            if x_uv is not None and x_uw is not None and x_uv != x_uw:
                continue
            if x_uv is None:
                for val in [0, 1]:
                    assignments[f'x_{u}_{v}'] = val
                    result = backtrack()
                    if result != float('inf'):
                        return result + 1
                    del assignments[f'x_{u}_{v}']
            if x_uw is None:
                for val in [0, 1]:
                    assignments[f'x_{u}_{w}'] = val
                    result = backtrack()
                    if result != float('inf'):
                        return result + 1
                    del assignments[f'x_{u}_{w}']
        return float('inf')

    start_time = time.time()
    result = backtrack()
    elapsed = time.time() - start_time
    if elapsed > 230:
        return float('inf')
    return result

def run_trial(seed):
    n = random.choice([8, 10, 12, 14, 16])

```

```

adj = sample_3_regular_graph(n, seed)
omega = sample_odd_charge(n, seed)
paths = find_simple_paths(adj, omega)
M3 = compute_M3(adj, omega, paths)
t_star = tseitin_tree_resolution(adj, omega)
metric_value = math.log2(t_star) / (n * M3) if M3 > 0 else float('inf')
metric_value2 = n**(2/3) * M3
conjecture_holds = metric_value >= 0.05 and metric_value2 >= 0.05
counterexample = f"n={n}, log2(t*)/(n*M3)={metric_value}, n^(2/3)*M3={metric_value2}" if not c
return {
    "metric_name": "log2(t*)/(n*M3)",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
    "n": n,
    "M3": M3,
    "t_star": t_star
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        trials.append(trial)

    metric_values = [trial["metric_value"] for trial in trials if trial["conjecture_holds"]]
    metric_values2 = [trial["n"]**(2/3) * trial["M3"] for trial in trials if trial["conjecture_holds"]]
    mean_metric = sum(metric_values) / len(metric_values) if metric_values else 0
    std_metric = math.sqrt(sum((x - mean_metric)**2 for x in metric_values) / len(metric_values))
    mean_metric2 = sum(metric_values2) / len(metric_values2) if metric_values2 else 0
    std_metric2 = math.sqrt(sum((x - mean_metric2)**2 for x in metric_values2) / len(metric_values2))
    support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)

    if all(trial["conjecture_holds"] for trial in trials):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not trial["conjecture_holds"] for trial in trials):
        first_failing_seed = next(trial["seed"] for trial in trials if not trial["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={trials[first_failing_seed]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=0")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e01c7e79.py", line 267, in <module>
    trial = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e01c7e79.py", line 243, in run_trial
    adj = sample_3_regular_graph(n, seed)
    ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e01c7e79.py", line 134, in sample_3_reg
    if is_connected(adj) and girth(adj) >= 4 and spectral_gap(adj) >= 0.4:
    ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e01c7e79.py", line 112, in spectral_gap
    L = matrix_sub(D, adj)
    ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e01c7e79.py", line 35, in matrix_sub
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]
    ~~~~~^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in matrix_sub during spectral_gap computation before any instances were evaluated, so no data on the two pre-registered metrics was produced. | next: Fix the matrix dimension bug in sample_3_regular_graph/spectral_gap (ensure adj is built as a full n×n matrix before Laplacian subtraction) and rerun the 120-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	224515
2	preregistration	claude_max	opus	0	0	8967
3	novelty	claude_max	opus	0	0	4305
4	novelty	claude_max	opus	0	0	8504
5	test_gen	mistral	codestral-latest	0	0	330018
6	test_gen	mistral	codestral-latest	0	0	333772
7	test_gen	mistral	codestral-latest	0	0	327618
8	test_gen	mistral	codestral-latest	0	0	333739
9	judge	claude_max	opus	0	0	4399

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1575837 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in research/audit/67ed17c63684.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/67ed17c63684.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/67ed17c63684.tar.gz` (if generated)